Technical Report

ShelfSense API

Module	XJCO3011 Web Services and Web Data	**Project**	ShelfSense API
Project Type	Individual Coursework	**Data Source**	Open Library
Repository	github.com/CwhyT/web_cw1	**API Style**	RESTful JSON API

Submission Materials

GitHub repository: Public repository with visible commit history: https://github.com/CwhyT/web_cw1
API documentation: https://github.com/CwhyT/web_cw1/tree/main/documentation
Presentation slides: https://github.com/CwhyT/web_cw1/tree/main/documentation
Deployment status: The current project is designed for local execution; no public hosted deployment link is included.
Authentication method: Bearer token authentication via POST /api/auth/login.
Test evidence: Current automated test summary: 16 passed.

1. Introduction

ShelfSense API is a RESTful web API for book discovery, reading list management, review storage, user preference analysis, and recommendation generation. The project was developed to satisfy the coursework requirement for a data-driven web service using HTTP endpoints, JSON responses, SQL-backed CRUD operations, documentation, testing, and version-controlled development.

The system combines public book metadata from Open Library with locally stored user activity. Open Library is used for live search and detail lookup, while local SQL data supports user accounts, reading lists, reviews, analytics, and recommendations. This hybrid design demonstrates both external API integration and relational data modelling in one coherent application.

2. Project Scope and Requirements Coverage

The project was designed around three connected use cases: discovering books from a public source, allowing users to manage personal reading lists and reviews, and analysing saved activity to generate preference insights and rule-based recommendations.

- SQL-backed relational data model
- multiple HTTP endpoints with JSON request/response bodies
- at least one full CRUD resource
- external data integration
- documentation and automated testing evidence

The final implementation goes beyond the minimum pass requirement because it includes two full CRUD resources, authentication, ownership checks, analytics, and recommendation functionality.

3. Technical Stack and Justification

3.1 FastAPI

FastAPI was selected because it is well suited to building RESTful JSON APIs quickly and clearly. Pydantic-based validation, Python type hints, and built-in Swagger UI / ReDoc make it effective both for implementation and for coursework demonstration.

3.2 SQLAlchemy and SQLite

SQLAlchemy provides a maintainable mapping between Python models and relational tables, which supports the coursework requirement for SQL-based CRUD functionality. SQLite was used during development because it is lightweight, easy to configure locally, and appropriate for a coursework-scale prototype. The design remains portable enough for migration to PostgreSQL in future work.

3.3 Open Library and Supporting Libraries

Open Library was chosen as the external data source because it provides public book metadata through a straightforward API and removes the need for a complete local catalogue. Supporting libraries include Pydantic for validation and serialisation, httpx for external HTTP requests, and pytest for automated testing.

4. System Design

4.1 Architectural Approach

The application uses a layered structure: routers define HTTP endpoints, models define SQLAlchemy entities, schemas validate request and response data, services encapsulate external API calls and recommendation logic, and tests verify behaviour. This separation improves maintainability and makes the design easier to explain.

4.2 RESTful API Design

The API uses resource-oriented endpoints and standard HTTP methods: GET for retrieval, POST for creation, PUT for update, and DELETE for deletion. JSON is used throughout, and conventional status codes such as 201 Created, 204 No Content, and 404 Not Found are returned where appropriate.

REST was chosen instead of GraphQL because the application is resource-oriented, CRUD-heavy, and easier to document, validate, and test within the coursework scope.

4.3 Data Source Strategy

Open Library is used for live search and book detail retrieval, while local SQLite storage is used for users, cached books, reading lists, and reviews. Selected books are cached only when needed, which keeps the public dataset external while making local analytics and user-owned state easier to manage with SQL queries.

5. Database Design

The main database tables are users, books, reading_lists, reading_list_items, and reviews.

- users store account and authentication information
- books store cached book records added to reading lists
- reading_lists store user-owned list containers
- reading_list_items store list–book relationships and reading status
- reviews store user ratings and review text for locally cached books

The main relationships are one user to many reading lists, one reading list to many reading list items, one user to many reviews, and one book to many reviews. This schema supports the project's two main CRUD resources and also enables joins for analytics.

6. Implemented Functionality

- Authentication: registration, login, current-user lookup, bearer-token protection, and ownership checks
- Book discovery: Open Library search, single-book detail retrieval, and work-key normalisation
- Reading lists: create, list, view, update, and delete reading lists
- Reading list items: add a book, locally cache it, update reading status, and remove it from a list
- Reviews: create, retrieve, update, and delete reviews, plus list all reviews for a local book
- Analytics and recommendations: genre frequency analysis, user preference analysis, and rule-based recommendations

These features mean the system exceeds the minimum CRUD requirement and adds analytical value beyond a basic prototype.

7. Security, Validation, and Error Handling

Protected endpoints require a bearer token obtained from the login endpoint. Ownership checks prevent users from creating or modifying another user's reading lists or reviews.

Validation is handled through Pydantic schemas, and the API returns simplified error messages to improve readability. Typical examples include "Invalid email format", "Missing required field: email", and "Rating must be between 1 and 5".

The system uses conventional HTTP status codes, including 401 Unauthorized for invalid authentication, 403 Forbidden for ownership violations, 404 Not Found for missing resources, 409 Conflict for duplicates, and 422 Unprocessable Entity for validation problems.

8. Testing Strategy and Evidence

Automated testing was performed with pytest and FastAPI's TestClient. External Open Library calls were mocked so that tests could run reliably without network dependence. The current automated test result is 16 passed.

- successful login and invalid-password rejection
- authentication requirement for /api/auth/me
- book-search and book-detail transformation logic
- reading-list creation and prevention of cross-user access
- review CRUD flow and missing-book rejection
- prevention of deleting another user's review
- analytics, recommendations, validation-error handling, and /health

In addition to automated tests, manual verification was supported by a plain-text guide (manual_test_template.txt) containing copyable PowerShell examples.

9. Challenges, Solutions, and Lessons Learned

Three practical challenges shaped the design. First, combining a public external dataset with a local SQL model required a clear boundary: Open Library is used only for live discovery, while selected books are cached locally once they are saved. Second, bearer-token authentication and ownership checks were added so that the API would represent real user data rather than an open CRUD prototype. Third, recommendations were implemented with a rule-based strategy rather than machine learning so that the feature remained useful while staying appropriate for coursework scope.

The project also highlighted broader lessons about API development. Hybrid external/local data architectures are often more effective than storing everything in one place; authentication and permissions are essential once user-owned state is introduced; and testing plus documentation are not secondary tasks, because they expose design inconsistencies that are easy to miss during coding alone.

10. Limitations and Future Improvements

The current project is designed primarily for local execution with SQLite, has no hosted deployment, and uses rule-based rather than machine-learning-based recommendations. It also has no frontend client, and live book search depends on Open Library availability.

If extended further, the most valuable next steps would be deployment to a public hosting platform, migration from SQLite to PostgreSQL, replacement of the lightweight token mechanism with JWT, richer analytics, and improved recommendation ranking and diversity.

11. Coursework Deliverables Status

At code level, the project already satisfies the major technical requirements: SQL-backed CRUD resources are implemented, more than four HTTP endpoints are available, JSON responses are used throughout, external API integration is present, automated testing evidence exists, and GitHub version history is available.

Supporting coursework documents have also been prepared, including the API documentation, README, presentation materials, and this technical report. Before final submission, the report PDF should explicitly reference the repository, the API documentation PDF, the presentation slides, the GenAI declaration and reflection, and the attached conversation-log appendix.

12. GenAI Declaration and Reflection

Generative AI tools were used in a declared and supportive capacity during this coursework. Their roles included brief interpretation, project planning, debugging support, repository review, documentation drafting, and identification of missing requirements or test gaps. AI output was not accepted uncritically: suggestions were manually reviewed, adapted where necessary, and verified through local testing and repository inspection before being retained.

The most useful contribution of GenAI was accelerating planning and documentation refinement while helping identify incomplete submission materials, authentication hardening tasks, and testing gaps. However, API behaviour, file paths, compliance with the brief, and repository state still required manual checking.

12.1 Declared GenAI Tools and Purposes

Tool	Purpose	Example Use in This Project	Human Verification
ChatGPT	brief interpretation, planning, writing support	analysing the coursework brief, structuring the report, and identifying missing deliverables	outputs were checked against the brief and rewritten where needed
Codex	repository assistance, implementation support, debugging, and test improvement	refining routes, validation messages, tests, repository cleanup, and documentation drafts	all code and document changes were inspected locally and tested before retention

12.2 Appendix Requirement

Representative conversation-log evidence should be attached as a supplementary appendix in the final submission package. Suitable evidence includes brief interpretation, topic and data-source selection, API design planning, debugging or validation refinement, and documentation review. This appendix should contain exported text excerpts, screenshots, or a separate appendix PDF referenced clearly from the final report.

13. Conclusion

ShelfSense API demonstrates a complete API development workflow from project scoping and database design to CRUD implementation, authentication, testing, analytics, and supporting documentation. It satisfies the core coursework expectations for a RESTful, SQL-backed web service and goes beyond the minimum requirement through recommendation features, access control, and structured testing. The current implementation is functional, documented, and ready to be packaged for final submission.

References

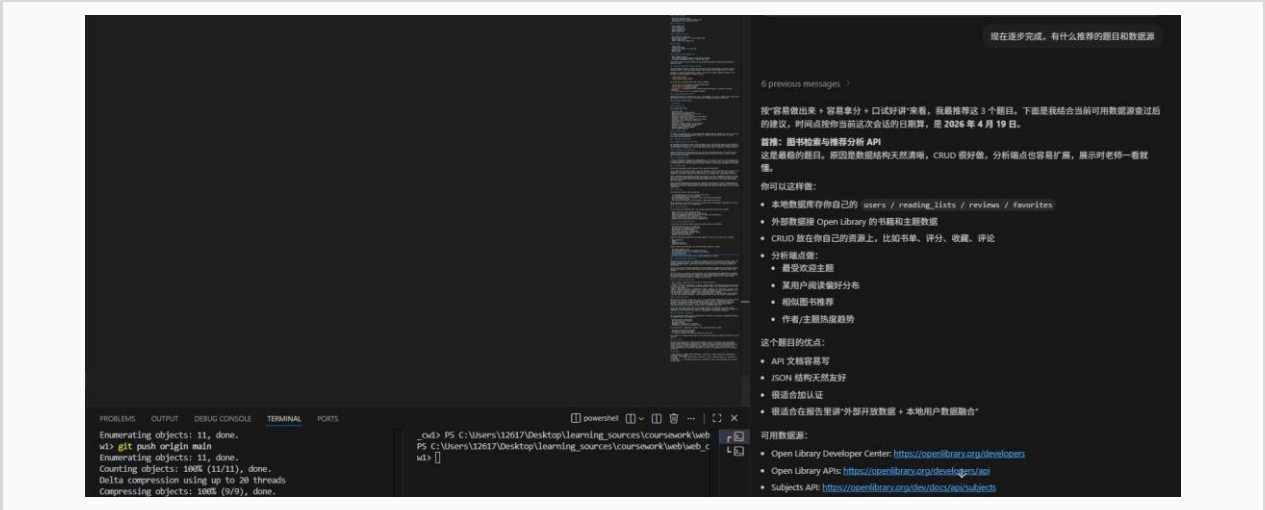
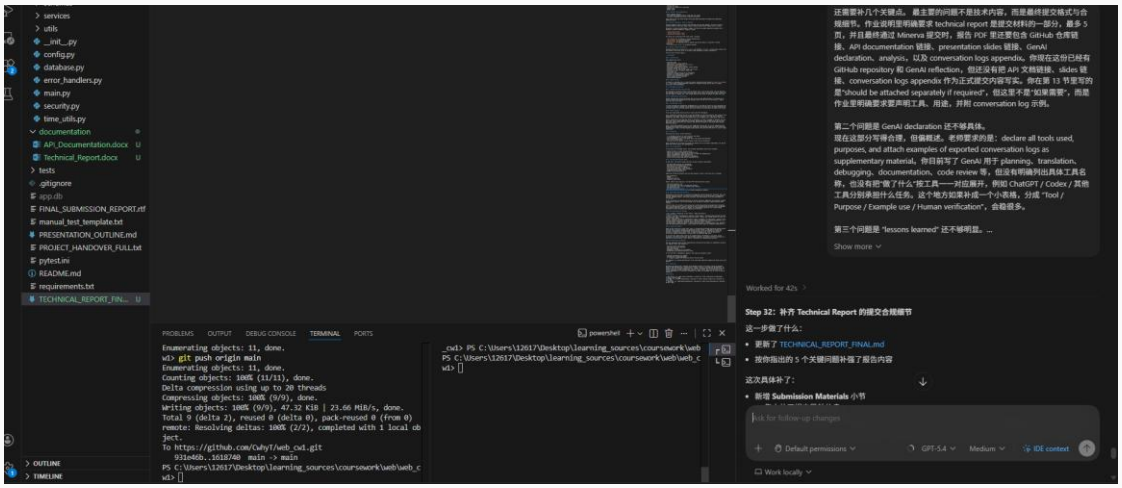
Open Library (n.d.) Open Library Developers. Available at: https://openlibrary.org/developers (Accessed: 20 April 2026).
FastAPI (n.d.) FastAPI Documentation. Available at: https://fastapi.tiangolo.com/ (Accessed: 20 April 2026).
SQLAlchemy (n.d.) SQLAlchemy Documentation. Available at: https://docs.sqlalchemy.org/ (Accessed: 20 April 2026).

Appendix A. Representative GenAI Conversation Evidence

Replace the placeholders below with exported text snippets or screenshots before final PDF submission. Keep only representative examples rather than full transcripts.

A1. Brief interpretation and requirements extraction

A2. API design / documentation refinement



A3. Debugging, testing, or submission-material review